

rateServoFullNonlinear

1 Executive Summary

This module implements a nonlinear rate servo control that uses the attitude steering message from the `mrpSteering` module and determines the control torque vector on the spacecraft.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
cmdTorqueOutMsg	<code>CmdTorqueBodyMsgPayload</code>	commanded torque output message
guidInMsg	<code>AttGuidMsgPayload</code>	attitude guidance input message
vehConfigInMsg	<code>VehicleConfigMsgPayload</code>	vehicle configuration input message
rwSpeedsInMsg	<code>RWSpeedMsgPayload</code>	(optional) RW speed input message
rwAvailInMsg	<code>RWAvailabilityMsgPayload</code>	(optional) RW availability input message
rwParamsInMsg	<code>RWArrayConfigMsgPayload</code>	(optional) RW configuration parameter input message
rateSteeringInMsg	<code>RateCmdMsgPayload</code>	commanded rate input message

3 Module Parameters

The following table lists all the module parameters than can be set. The parameters are optional unless indicated (if not specified default is used).

Table 2: Module Parameters

Parameter Name	Default	Description	Bounds
P	0	Rate error feedback gain	Must not be negative (checked in setter)
Ki	0	Integral feedback gain	If negative, no integral feedback is applied and the corresponding computation is skipped
integralLimit	0	Limit for integral feedback term (term will be capped by <code>integralLimit</code>)	Must not be negative (checked in setter)
knownTorquePntB.B	[0, 0, 0]	Known external torque in body frame components	None

4 Module Assumptions and Limitations

This control assumes the spacecraft is rigid.

5 Initialization

The module is configured by:

```
module = rateServoFullNonlinear.RateServoFullNonlinear()
module.modelTag = "rateServoFullNonlinear"
module.P = P
module.Ki = Ki
module.integralLimit = integral_limit
module.knownTorquePntB_B = known_torque
```

6 Detailed Module Description

6.1 Overview

This module computes a commanded control torque vector $\mathbf{L}_r$ using a rate based steering law that drives a body frame $\mathcal{B} : \{\hat{\mathbf{B}}_1, \hat{\mathbf{B}}_2, \hat{\mathbf{B}}_3\}$ towards a time varying reference frame $\mathcal{R} : \{\hat{\mathbf{R}}_1, \hat{\mathbf{R}}_2, \hat{\mathbf{R}}_3\}$, based on a desired reference frame $\mathcal{B}^* : \{\hat{\mathbf{B}}_1, \hat{\mathbf{B}}_2, \hat{\mathbf{B}}_3\}$ (the desired body frame from the kinematic steering law).

The output message is a body-frame control torque vector. The required attitude guidance message contains both attitude tracking error rates as well as reference frame rates. This message is read in with every update cycle. The vehicle configuration message is only read in on reset and contains the spacecraft inertia tensor about the vehicle's center of mass location. The commanded body rates are read in from the steering module output message.

The reaction wheel (RW) configuration message is optional. If the message name is specified, then the RW message is read in, otherwise the inertia of the wheels is not considered. If the optional RW availability message is present, then the control will only use the RWs that are marked available.

The servo rate feedback control can compensate for Reaction Wheel (RW) gyroscopic effects as well. This is an optional input message where the RW configuration array message contains the RW spin axis $\hat{g}_{s,i}$ information and the RW polar inertia about the spin axis $I_{Ws,i}$. This is only read in on reset. The RW speed message contains the RW speed Ω_i and is read in every time step. The optional RW availability message can be used to include or not include RWs in the MRP feedback. This allows the module to selectively turn off some RWs. The default is that all RWs are operational and are included.

6.2 Steering Law Goals

This technical note develops a rate based steering law that drives a body frame $\mathcal{B} : \{\hat{\mathbf{B}}_1, \hat{\mathbf{B}}_2, \hat{\mathbf{B}}_3\}$ towards a time varying reference frame $\mathcal{R} : \{\hat{\mathbf{R}}_1, \hat{\mathbf{R}}_2, \hat{\mathbf{R}}_3\}$. The inertial frame is given by $\mathcal{N} : \{\hat{\mathbf{N}}_1, \hat{\mathbf{N}}_2, \hat{\mathbf{N}}_3\}$. The RW coordinate frame is given by $\mathcal{W} : \{\hat{\mathbf{g}}_{s,i}, \hat{\mathbf{g}}_{t,i}, \hat{\mathbf{g}}_{g,i}\}$. Using MRPs, the overall control goal is

$$\sigma_{\mathcal{B}/\mathcal{R}} \rightarrow 0$$

The reference frame orientation $\sigma_{\mathcal{R}/\mathcal{N}}$, angular velocity $\omega_{\mathcal{R}/\mathcal{N}}$ and inertial angular acceleration $\dot{\omega}_{\mathcal{R}/\mathcal{N}}$ are assumed to be known.

6.3 Rotational Dynamics

The rotational equations of motion of a rigid spacecraft with N Reaction Wheels (RWs) attached are given by

$$[I_{RW}]\dot{\omega} = -[\tilde{\omega}]([I_{RW}]\omega + [G_s]\mathbf{h}_s) - [G_s]\mathbf{u}_s + \mathbf{L}$$

where the inertia tensor $[I_{RW}]$ is defined as

$$[I_{RW}] = [I_s] + \sum_{i=1}^N (J_{t_i} \hat{\mathbf{g}}_{t_i} \hat{\mathbf{g}}_{t_i}^T + J_{g_i} \hat{\mathbf{g}}_{g_i} \hat{\mathbf{g}}_{g_i}^T)$$

The spacecraft inertia without the N RWs is $[I_s]$, while J_{s_i} , J_{t_i} and J_{g_i} are the RW inertias about the body fixed RW axis $\hat{\mathbf{g}}_{s_i}$ (RW spin axis), $\hat{\mathbf{g}}_{t_i}$ and $\hat{\mathbf{g}}_{g_i}$. The $3 \times N$ projection matrix $[G_s]$ is then defined as

$$[G_s] = [\cdots \hat{\mathbf{g}}_{s_i} \cdots]$$

The RW inertial angular momentum vector $\mathbf{h}_s$ is defined as

$$h_{s_i} = J_{s_i}(\omega_{s_i} + \Omega_i)$$

Here Ω_i is the i^{th} RW spin relative to the spacecraft, and the body angular velocity is written in terms of body and RW frame components as

$$\omega = \omega_1 \hat{\mathbf{b}}_1 + \omega_2 \hat{\mathbf{b}}_2 + \omega_3 \hat{\mathbf{b}}_3 = \omega_{s_i} \hat{\mathbf{g}}_{s_i} + \omega_{t_i} \hat{\mathbf{g}}_{t_i} + \omega_{g_i} \hat{\mathbf{g}}_{g_i}$$

6.4 Angular Velocity Servo Sub-System

To implement the kinematic steering control, a servo sub-system is included which will produce the required torques to make the actual body rates track the desired body rates. The angular velocity tracking error vector is defined as

$$\delta\omega = \omega_{\mathcal{B}/\mathcal{B}^*} = \omega_{\mathcal{B}/\mathcal{N}} - \omega_{\mathcal{B}^*/\mathcal{N}}$$

where the $\mathcal{B}^*$ frame is the desired body frame from the kinematic steering law. Note that

$$\omega_{\mathcal{B}^*/\mathcal{N}} = \omega_{\mathcal{B}^*/\mathcal{R}} + \omega_{\mathcal{R}/\mathcal{N}}$$

where $\omega_{\mathcal{R}/\mathcal{N}}$ is obtained from the attitude navigation solution, and $\omega_{\mathcal{B}^*/\mathcal{R}}$ is the kinematic steering rate command. To create a rate-servo system that is robust to unmodeld torque biases, the state $\mathbf{z}$ is defined as:

$$\mathbf{z} = \int_{t_0}^{t_f} \delta\omega dt$$

Let $[P]^T = [P]$ be a symmetric positive definite rate feedback gain matrix. The servo rate feedback control is defined as

$$\begin{aligned} [G_s]\mathbf{u}_s &= [P]\delta\omega + [K_I]\mathbf{z} - [\tilde{\omega}_{\mathcal{B}^*/\mathcal{N}}]([I_{RW}]\omega_{\mathcal{B}/\mathcal{N}} + [G_s]\mathbf{h}_s) \\ &\quad - [I_{RW}](\omega'_{\mathcal{B}^*/\mathcal{R}} + \dot{\omega}_{\mathcal{R}/\mathcal{N}} - \omega_{\mathcal{B}/\mathcal{N}} \times \omega_{\mathcal{R}/\mathcal{N}}) + \mathbf{L} \end{aligned}$$

Defining the right-hand-side as $-\mathbf{L}_r$, this is rewritten in compact form as

$$[G_s]\mathbf{u}_s = -\mathbf{L}_r$$

The `rateServoFullNonlinear` module writes the control torque $\mathbf{L}_r$ to the `CmdTorqueBodyMsgPayload` output message.

Downstream, the `rwMotorTorque` module can then be used to compute the array of RW motor torques by solving the typical minimum norm inverse:

$$\mathbf{u}_s = [G_s]^T ([G_s][G_s]^T)^{-1} (-\mathbf{L}_r)$$

7 Additional Information

The value of `integralLimit`, used to limit the degree of integrator windup and reduce the chance of controller saturation. The integrator is required to maintain asymptotic tracking in the presence of an external disturbing torque. For this module, the integration limit is applied to each element of the integrated error vector z , and any elements greater than the limit are set to the limit instead.